

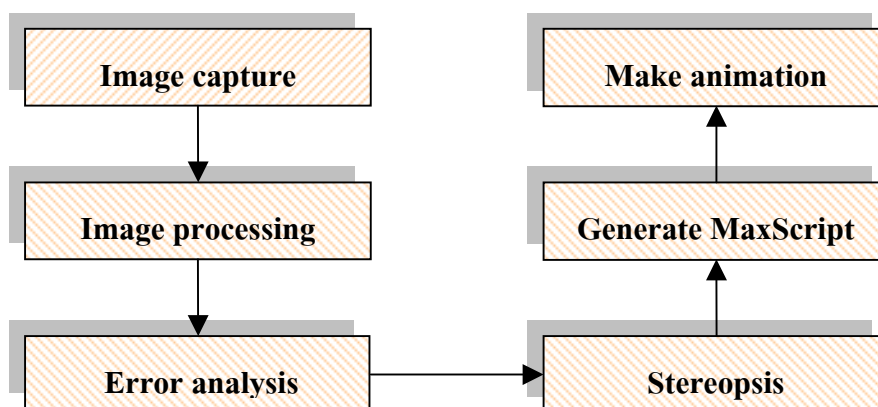
## บทที่ 7

### การออกแบบโปรแกรมจับรูปแบบการเคลื่อนไหว

#### 7.1 ภาพรวมของโครงการ

โปรแกรมจับรูปแบบการเคลื่อนไหว (Motion capture) เป็นโปรแกรมที่ใช้ในการจับการเคลื่อนไหวของสิ่งมีชีวิตแล้วนำไปแมชกับตัวการ์ตูนสามมิติให้ตัวการ์ตูนนั้นมีการเคลื่อนไหวเหมือนกับการเคลื่อนไหวของสิ่งมีชีวิตต้นแบบที่เราทำการบันทึกมา โดยผลลัพธ์จะอยู่ในรูปของกราฟิก 3 มิติ ซึ่งโปรแกรมจับรูปแบบการเคลื่อนไหวนี้สามารถนำไปใช้ประโยชน์ได้อย่างกว้างขวางเช่นนำไปช่วยในการทำภาพยนตร์ที่มีตัวละครถูกสร้างขึ้นจากโปรแกรม 3 มิติเพื่อให้ตัวละครนั้นมีการเคลื่อนไหวที่สมจริงหรือนำไปใช้ในวงการกีฬาเพื่อใช้ศึกษาทฤษฎีการเคลื่อนไหวของข้อต่อต่างๆเพื่อเพิ่มประสิทธิภาพการเคลื่อนไหวให้กับนักกีฬาหรือนำไปใช้ในวงการแพทย์หรือนำไปช่วยในการสร้างเกมให้ตัวละครในเกมมีการเคลื่อนไหวที่สมจริงมากขึ้นเพื่อให้ผู้เล่นมีความสุขในการเล่นเกมนมากขึ้น

ในการออกแบบและการเขียนโปรแกรมจับรูปแบบการเคลื่อนไหวนี้ จะมีรูปแบบขั้นตอนในการทำงานดังรูปที่ 7.1



รูปที่ 7.1 แสดงภาพโดยรวมของระบบ

## 7.2 การออกแบบและการเขียนโปรแกรม

การออกแบบโปรแกรมจับรูปแบบการเคลื่อนไหวนั้นเราได้แบ่งออกเป็นส่วนที่สำคัญๆทั้งหมด 6 ส่วนด้วยกันดังที่ได้แสดงไว้แล้วในรูปที่ 7.1 ซึ่งแต่ละส่วนจะมีการทำงานที่แตกต่างกันไป โดยเราจะทำการอธิบายแต่ละส่วนดังนี้

### 7.2.1 การบันทึกภาพเคลื่อนไหว (Image capture)

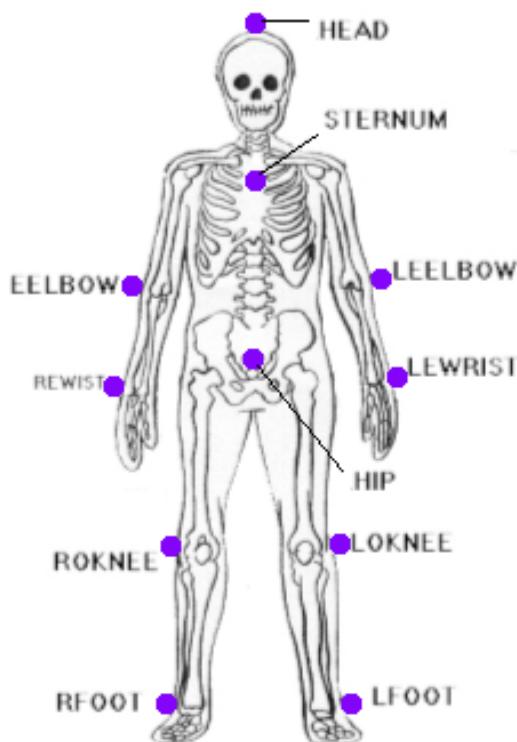
ส่วนของการบันทึกภาพเคลื่อนไหว นี่จะเป็นในส่วนของอุปกรณ์ คือการใช้กล้องในการจับภาพการเคลื่อนไหวของสิ่งมีชีวิตโดยในส่วนนี้เราได้ใช้กล้องเพียงแค่ 2 ตัวในการบันทึก ส่วนของอุปกรณ์มาร์เกอร์นั้นจากการทดลองเพื่อหาอุปกรณ์สะท้อนแสงที่เหมาะสมที่สุดมาใช้ในโครงการนี้ ในที่สุดเราได้เลือกใช้ลูกปัดปองที่ทำการติดหลอดไฟขนาด 3 โวลต์ ไว้ภายใน ซึ่งแสดงไว้ดังรูปที่

7.2



รูปที่ 7.2 แสดงอุปกรณ์มาร์เกอร์ที่ใช้ติดตามส่วนต่างๆของร่างกาย

ส่วนของการติดอุปกรณ์มาร์เกอร์ตามส่วนต่างๆของร่างกายนั้น การที่จะติดตรงส่วนไหนบ้างนั้นขึ้นอยู่กับว่าเราต้องการตรวจจับตรงส่วนไหนบ้าง จากโครงการนี้เราจะทำการติดมาร์เกอร์ตรงส่วนสำคัญหลักๆเท่านั้น ซึ่งในการจับการเคลื่อนไหวนี้จะมีจุดจำกัดการเคลื่อนไหวอยู่บ้างเพราะเราใช้กล้องเพียงแค่ 2 ตัวเท่านั้น ดังนั้นการเคลื่อนไหวในท่าทางที่ทำให้มาร์เกอร์ถูกบังหรือซ้อนทับกันนั้นจะทำให้ไม่สามารถตรวจจับมาร์เกอร์ตัวที่ถูกบังนั้นได้ ทำให้ไม่รู้ว่าขณะนั้นมาร์เกอร์ที่ถูกบังนั้นอยู่ ณ ตำแหน่งไหนของภาพ และข้อจำกัดอีกข้อหนึ่งคือมันไม่สามารถตรวจจับการหมุนของร่างกายส่วนต่างๆได้เพราะเนื่องมาจากการใช้กล้องเพียงแค่ 2 ตัว รูปที่ 7.3 แสดงการติดอุปกรณ์มาร์เกอร์ตามส่วนต่างๆของร่างกายที่ได้ทำในการทดลองนี้



รูปที่ 7.3 แสดงตำแหน่งที่เราทำการติดอุปกรณ์มาร์กเกอร์

ส่วนของไฟล์ที่ได้จากการบันทึกภาพเคลื่อนไหวนี้จะอยู่ในรูปของฟอร์แมตไฟล์วีดีโอเอวีไอ ซึ่งจะได้ไฟล์อยู่ 2 ไฟล์คือไฟล์จากกล้องทั้งสองตัวคือกล้องทางด้านซ้ายและกล้องทางด้านขวา ในโครงการนี้เราจะใช้ไฟล์วีดีโอที่มีขนาด 320 x 240 พิกเซล และความละเอียดของสีจะใช้นขนาด 24 บิต ส่วนอัตราเฟรมเรทอยู่ที่ 25 เฟรมต่อวินาที ส่วนของอัตราเฟรมเรทนี้ถือว่าต่ำมากซึ่งขึ้นอยู่กับชนิดของกล้องที่เราใช้ด้วย ควรจะเลือกใช้กล้องที่มีอัตราเฟรมเรทที่สูงๆ ยิ่งอัตราเฟรมเรทยิ่งสูงยิ่งดี

ส่วนของสถานที่ที่จะใช้นั้นควรจะใช้สถานที่ ที่ไม่มีสิ่งที่จะมารบกวนในการบันทึกภาพ เช่นแสงไฟต่างๆ และสถานที่นั้นควรจะต้องมีโต๊ะหรือเก้าอี้หรือไม่ว่าสภาพแวดล้อมควรจะเป็นสีดำเพราะว่าโครงการของเราจะใช้ตรวจจับแสงสีขาวที่ได้จากมาร์กเกอร์

#### 7.2.2 อิมเมจโปรเซสซิง (Image processing)

ในส่วนนี้จะทำหน้าที่ในการนำไฟล์วีดีโอที่ได้มาประมวลผลเพื่อทำการหาดำแหน่งของมาร์กเกอร์ที่ปรากฏในภาพเพื่อนำตำแหน่งที่ได้ส่งไปให้ส่วนอื่นทำงานต่อไป โดยในส่วนนี้อิมเมจโปรเซสซิงจะมีขั้นตอนการทำงานหลักๆอยู่ 4 ขั้นตอนดังนี้

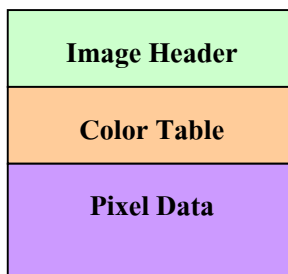
- ส่วนของการจัดการกับไฟล์วีดีโอเอวีไอ
- ส่วนของการหาดำแหน่งของมาร์กเกอร์
- ส่วนของการจัดกลุ่มตำแหน่งพิกเซลของมาร์กเกอร์

### - ส่วนของการจัดลำดับมาร์เกอร์

โดยแต่ละส่วนจะมีความเรียงตามลำดับกันไป และเราจะทำการอธิบายการทำงานของแต่ละส่วนโดยละเอียดในหัวข้อต่อไป

#### 7.2.2.1 ส่วนของการจัดการกับไฟล์วิดีโอเอวีไอ

การทำงานในส่วนนี้คือการที่จะนำไฟล์วิดีโอ ที่ได้จากกล้องทั้งสองตัวมาทำให้อยู่ในรูปของดีไอบี(Device Independence bitmap (DIB) ) ซึ่งฟอร์แมตไฟล์วิดีโอจะมีส่วนของดีไอบี อยู่ด้วย โดย ดีไอบี นี้จะอยู่ในส่วนของเฟรมของไฟล์วิดีโอ ดีไอบีจะเป็นส่วนในการเก็บข้อมูลของภาพในแต่ละเฟรม โดยสามารถไปดูฟอร์แมตของดีไอบีแสดงได้ดังภาพที่ 7.4 เราจะทำการเขียนโปรแกรมเพื่อใช้ในการเข้าไปดึงข้อมูลในดีไอบีเพื่อที่ได้ข้อมูลของภาพในเฟรมนั้นๆออกมา เพื่อที่จะนำข้อมูลนั้นส่งไปให้ในส่วนต่อไปทำงาน



รูปที่ 7.4 แสดง Format ของเฟรม AVI

#### 7.2.2.2 ส่วนของการหาตำแหน่งของมาร์เกอร์

ในส่วนนี้จะทำการรับข้อมูลดีไอบีที่ถูกส่งมาเพื่อนำไปหาตำแหน่งของมาร์เกอร์ โดยวิธีที่ใช้ในการหาตำแหน่งของมาร์เกอร์นั้น เราจะเข้าไปสแกนหาทุกตำแหน่งของพิกเซลในภาพที่มีค่าความเข้มแสงตรงตามที่เรากำลังต้องการ และทำการเก็บตำแหน่งทุกพิกเซลที่สแกนหาได้ไว้และทำการส่งตำแหน่งที่หาได้นี้ไปให้ส่วนต่อไปทำงานโดยมันจะมีความทำงานอย่างนี้ทุกครั้งไปเรื่อยๆในทุกๆเฟรม

#### 7.2.2.3 ส่วนของการจัดกลุ่มตำแหน่งพิกเซลของมาร์เกอร์

ส่วนนี้จะนำข้อมูลที่หาได้จากส่วนที่แล้วมาทำการหากลุ่มของพิกเซลที่คาดว่าจะ เป็นกลุ่มของมาร์เกอร์ตัวเดียวกันมาทำการจัดเป็นกลุ่มๆของมาร์เกอร์แต่ละตัว แล้วเราจะทำการเช็ค ว่าพอแบ่งเป็นกลุ่มแล้ว ได้จำนวนของกลุ่มเท่ากับจำนวนของมาร์เกอร์หรือไม่ถ้าไม่ก็จะกลับไปทำงานในขั้นตอนที่ 2 ใหม่มันก็จะทำการวนการทำงานแบบนี้ไปเรื่อยๆไปจนถึงค่าหนึ่งถ้ายังไม่ได้จำนวนกลุ่มเท่ากับจำนวนของมาร์เกอร์ก็จะออกจากลูปแล้วแจ้งข้อผิดพลาดให้ผู้ใช้งานทราบ เมื่อเราได้จำนวนกลุ่มพิกเซลเท่ากับจำนวนมาร์เกอร์แล้ว เราก็จะส่งข้อมูลที่ได้นี้ไปให้ส่วนสุดท้ายทำงานต่อไป

#### 7.2.2.4 ส่วนของการจัดลำดับมาร์เกอร์

ส่วนนี้จะนำข้อมูลที่ได้มาทำการหาจุดศูนย์กลางของมาร์เกอร์แต่ละตัว แล้วทำการนำตำแหน่งศูนย์กลางที่ได้ ไปทำการเปรียบเทียบกับตำแหน่งมาร์เกอร์ของเฟรมที่แล้วเพื่อทำการตรวจเช็คค่าตำแหน่งมาร์เกอร์ในเฟรมปัจจุบันนั้นเป็นตำแหน่งมาร์เกอร์จุดไหนของร่างกาย เพื่อที่จะได้นำไปประมวลผลได้ถูกต้อง

#### 7.2.3 การวิเคราะห์ค่าความผิดพลาด (Error analysis)

การวิเคราะห์ค่าความผิดพลาด คือการที่เราได้ตำแหน่งพิกเซล X และ Y ของจำนวนเฟรมที่เรากำหนด ในโครงงานนี้เราใช้ 10 เฟรมต่อการปรับปรุงค่าที่ผิดพลาดในระหว่าง 10 เฟรมนี้เพื่อนำไปกำหนดเป็นคีย์เฟรม (Key frame) ซึ่งในกระบวนการวิเคราะห์และทำการปรับปรุงนี้เราจะทำการหาอัตราเฉลี่ยในการเคลื่อนที่ในแนวแกน X และ Y เพื่อเป็นการลดค่าผิดพลาดที่เกิดขึ้นจากการบันทึกภาพ ดังนั้นอย่างที่บอกเอาไว้แล้วว่ายิ่งอัตราเฟรมเรทของกล้องที่ใช้ยิ่งสูงเท่าไรก็ยิ่งดีเพราะจะทำให้มีข้อมูลที่นำมาวิเคราะห์หาค่าผิดพลาดมีมากขึ้นในระยะเวลาที่สั้นลง ทำให้เราสามารถบันทึกภาพเคลื่อนไหวที่มีการเคลื่อนไหวที่รวดเร็วและนำมาทำภาพแอนิเมชันที่มีการเคลื่อนไหวที่รวดเร็วนั้นได้ ซึ่งการใช้กล้องที่มีอัตราเฟรมเรทที่ต่ำไม่สามารถทำภาพแอนิเมชันที่มีการเคลื่อนไหวที่รวดเร็วได้ ซึ่งจากโครงงานนี้กล้องที่เราใช้นั้นมีอัตราเฟรมเรทที่ต่ำดังนั้นเราไม่สามารถบันทึกภาพการเคลื่อนไหวที่รวดเร็วแล้วนำมาทำภาพแอนิเมชันที่มีการเคลื่อนไหวที่รวดเร็วนั้นได้ การวิเคราะห์และปรับปรุงค่าผิดพลาดนั้นสามารถดูได้ในบทที่ 8 เรื่องผลการทดลองและการทดสอบ

#### 7.2.4 สเตอริโอออปซิส (Stereopsis)

สเตอริโอออปซิสเป็นกระบวนการที่ทำให้เราสามารถอ้างอิงตำแหน่งของวัตถุในรูปสามมิติได้โดยพิจารณาภาพอย่างน้อย 2 ภาพโดยภาพเหล่านั้นถ่ายในมุมมองที่แตกต่างกัน ซึ่งเราได้พูดถึงไปแล้วในบทที่ 4 และในหัวข้อนี้จะพูดถึงการนำทฤษฎีนี้มาใช้งาน

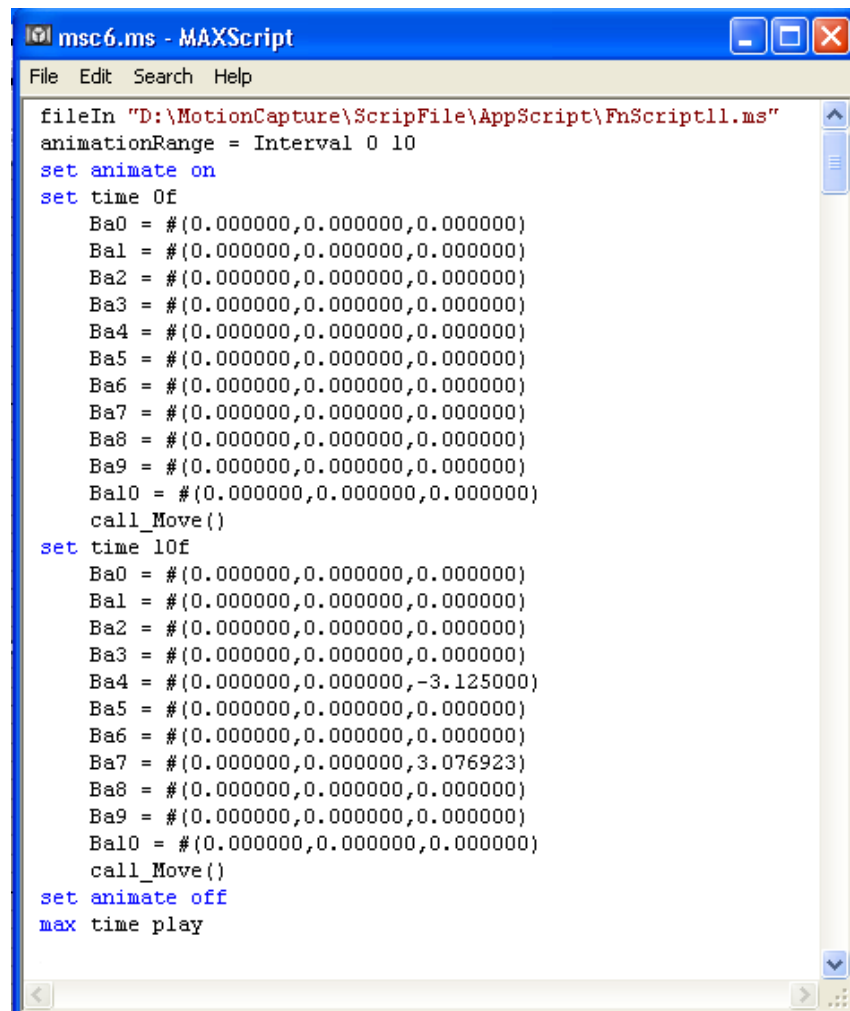
ในส่วนของการทำงานในสเตอริโอออปซิสนี้ จากรูปที่ 7.1 มันจะได้รับข้อมูลตำแหน่งการเคลื่อนที่ของมาร์เกอร์ในรูปของตำแหน่งพิกเซลที่ผ่านขั้นตอนการวิเคราะห์และปรับปรุงค่าผิดพลาดแล้ว โดยข้อมูลนี้จะอยู่ในรูปของตำแหน่งพิกเซล X และ Y ของมาร์เกอร์แต่ละตัว เราจะนำตำแหน่งที่ได้นี้ไปคำนวณหาตำแหน่งของมาร์เกอร์ที่อยู่ในรูปของคาทิเซียนคอดิเนต (Cartesian coordination)

#### 7.2.5 การสร้างไฟล์แมกซ์สคลิป (Generate MaxScript)

ไฟล์สคลิปคือภาษาสคลิปที่สร้างขึ้นสำหรับโปรแกรมสามมิติ สตูดิโอ แมกซ์ (3D Studio max) เพื่อนำมาใช้ในงานหลายๆอย่างในโปรแกรมสามมิติสตูดิโอแมกซ์เอง ซึ่งอย่างหนึ่งที่ภาษาแมกซ์สคลิปสามารถทำได้คือการทำแอนิเมชัน เราจึงนำความสามารถตรงส่วนนี้ของแมกซ์สคลิปมาใช้ในโครงงานนี้

ในการสร้างไฟล์สคลิปนี้จะถูกสร้างขึ้นเองโดยโปรแกรมของเรา โดยมันจะทำการรับข้อมูลตำแหน่งที่ได้จากสเตอริโอออปซิสมาทำการเขียนเป็นโค้ดแมกซ์สคลิปโดยในกระบวนการทำนี้เราจะทำการคูณตำแหน่งที่ได้คือในแนวแกน X , Y และ Z ด้วยค่า 10 เพราะว่าเป็นโปรแกรมสามมิติ สตูดิโอแมกซ์ การเคลื่อนที่ 1 หน่วยจะเท่ากับ 10 สมมุติเรากำหนดให้มีหน่วยเป็นนิ้ว ถ้าเราต้องการเคลื่อนที่เป็น

ระยะทาง 1 นิ้วในโปรแกรมสามมิติ สตูดิโอ แมกซ์ เราต้องทำการกำหนดให้มีค่าการเคลื่อนที่เท่ากับ 10 ดังนั้นนี่จึงเป็นเหตุผลที่ต้องคูณ 10 เข้าไป รูปที่ 7.5 แสดงตัวอย่างโค้ดแมกซ์สคริปต์ที่ได้จากการสร้างจากโปรแกรมที่เราเขียนขึ้น



```

fileIn "D:\MotionCapture\ScriptFile\AppScript\FnScript11.ms"
animationRange = Interval 0 10
set animate on
set time 0f
    Ba0 = #(0.000000,0.000000,0.000000)
    Ba1 = #(0.000000,0.000000,0.000000)
    Ba2 = #(0.000000,0.000000,0.000000)
    Ba3 = #(0.000000,0.000000,0.000000)
    Ba4 = #(0.000000,0.000000,0.000000)
    Ba5 = #(0.000000,0.000000,0.000000)
    Ba6 = #(0.000000,0.000000,0.000000)
    Ba7 = #(0.000000,0.000000,0.000000)
    Ba8 = #(0.000000,0.000000,0.000000)
    Ba9 = #(0.000000,0.000000,0.000000)
    Ba10 = #(0.000000,0.000000,0.000000)
    call_Move()
set time 10f
    Ba0 = #(0.000000,0.000000,0.000000)
    Ba1 = #(0.000000,0.000000,0.000000)
    Ba2 = #(0.000000,0.000000,0.000000)
    Ba3 = #(0.000000,0.000000,0.000000)
    Ba4 = #(0.000000,0.000000,-3.125000)
    Ba5 = #(0.000000,0.000000,0.000000)
    Ba6 = #(0.000000,0.000000,0.000000)
    Ba7 = #(0.000000,0.000000,3.076923)
    Ba8 = #(0.000000,0.000000,0.000000)
    Ba9 = #(0.000000,0.000000,0.000000)
    Ba10 = #(0.000000,0.000000,0.000000)
    call_Move()
set animate off
max time play

```

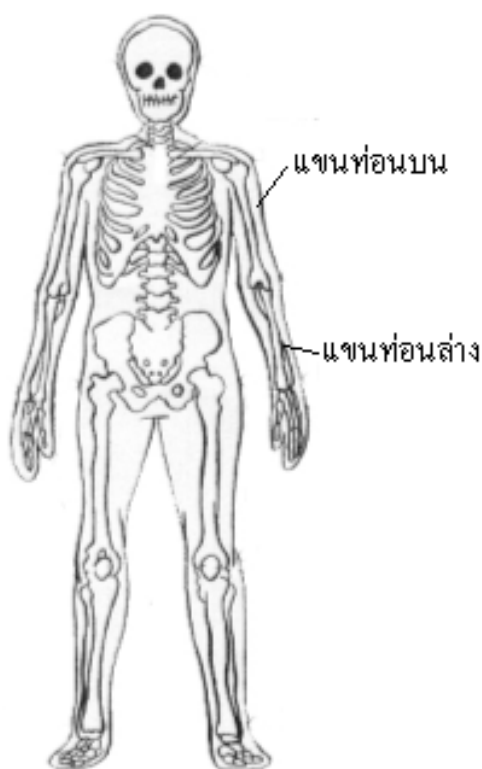
รูปที่ 7.5 แสดงตัวอย่างโค้ดแมกซ์สคริปต์

จากรูปที่ 7.5 สามารถอธิบายแต่ละส่วนของโค้ดแมกซ์สคริปต์ได้ดังนี้ บรรทัดแรกคือการเรียกใช้งานไฟล์ FnScript11.ms โดยเราจะพุดถึงไฟล์ตัวนี้ภายหลัง บรรทัดที่สองคือการเซตจำนวนเฟรมในการทำแอนิเมชัน จากตัวอย่างเป็นการเซตให้มีจำนวนเฟรม 293 เฟรมคือตั้งแต่เฟรมที่ 0 ถึงเฟรมที่ 292 บรรทัดที่สามคือการเซตเพื่อบอกว่าต่อไปนี่คือการทำแอนิเมชัน และบรรทัดที่สี่คือการเซตคีย์เฟรมที่ 0 บรรทัดที่ห้าถึงบรรทัดที่ สิบห้าคือการเซตระยะทางการเคลื่อนที่ในคีย์เฟรมที่ 0 ให้กับตัวแปร โดยตัวแปร Ba0 ถึง Ba10 หมายถึงตัวมาครเกอร์ทั้ง 11 จุด บรรทัดที่สิบหกคือการเรียกใช้ฟังก์ชัน call\_Move() ซึ่งเป็น

ฟังก์ชันที่อยู่ในไฟล์ FnScript11.ms ที่เราเรียกใช้ ส่วนสองบรรทัดสุดท้ายคือการบอกว่าสิ้นสุดการทำแอนิเมชันและทำการเรียกให้โปรแกรมทำการเล่นภาพแอนิเมชัน 3 มิติที่ได้สร้างไว้

#### 7.2.6 การทำแอนิเมชัน (Make animation)

ในส่วนของการทำแอนิเมชันนี้เป็นส่วนที่สำคัญอย่างหนึ่งในการทำโครงงานนี้ ใ้ว่าเราได้ตำแหน่งการเคลื่อนที่ของมาร์เกอร์ทุกจุดมาแล้วจะสามารถนำมาทำเป็นภาพแอนิเมชันได้เลย เพราะว่าการเคลื่อนไหวของสิ่งมีชีวิตอย่างเช่นมนุษย์ทุกส่วนของร่างกายจะมีความสัมพันธ์ต่อกัน จากรูปที่ 7.6 แสดงโครงสร้างของร่างกายมนุษย์ เมื่อเราทำการขยับแขนท่อนบนแขนท่อนล่างก็จะขยับตามไปด้วย ดังนั้นเมื่อเราเอาตำแหน่งการเคลื่อนที่ของมาร์เกอร์ที่ติดตรงแขนไปทำเป็นภาพแอนิเมชัน เราก็จะได้ตำแหน่งการเคลื่อนที่ของแขนท่อนบนและแขนท่อนล่างเมื่อเราเอาตำแหน่งการเคลื่อนที่ของแขนท่อนบนที่ได้ไปทำแอนิเมชันแขนท่อนล่างก็จะเคลื่อนที่ตามต่อจากนั้นเราก็ทำแอนิเมชันของแขนท่อนล่าง ซึ่งความจริงแขนท่อนล่างเคลื่อนที่ไปแล้วตามแขนท่อนบนพอเราเมาเซ่ดให้มันเคลื่อนที่อีกก็จะทำให้ผิดไป ดังนั้นเราจึงสร้างไฟล์แมกซ์สคลิปขึ้นมาไฟล์หนึ่งเพื่อช่วยแก้ปัญหาข้อนี้



รูปที่ 7.6 แสดงโครงสร้างร่างกายของมนุษย์

ไฟล์ที่ใช้ในการแก้ปัญหาที่เกิดขึ้นครั้งนี้มีชื่อไฟล์ว่า FnScript11.ms โดยเราสามารถอธิบายโค้ดของไฟล์นี้ได้ดังนี้

```

-----Variable-----
a0 = $'root'
a1 = $'BackBone3'
a2 = $'ArmDown-R'
a3 = $'ArmDown-L'
a4 = $'Hand-R'
a5 = $'Hand-L'
a6 = $Head
a7 = $'LegDownR'
a8 = $'LegDownL'
a9 = $'FootUpR'
a10 = $'FootUpL'

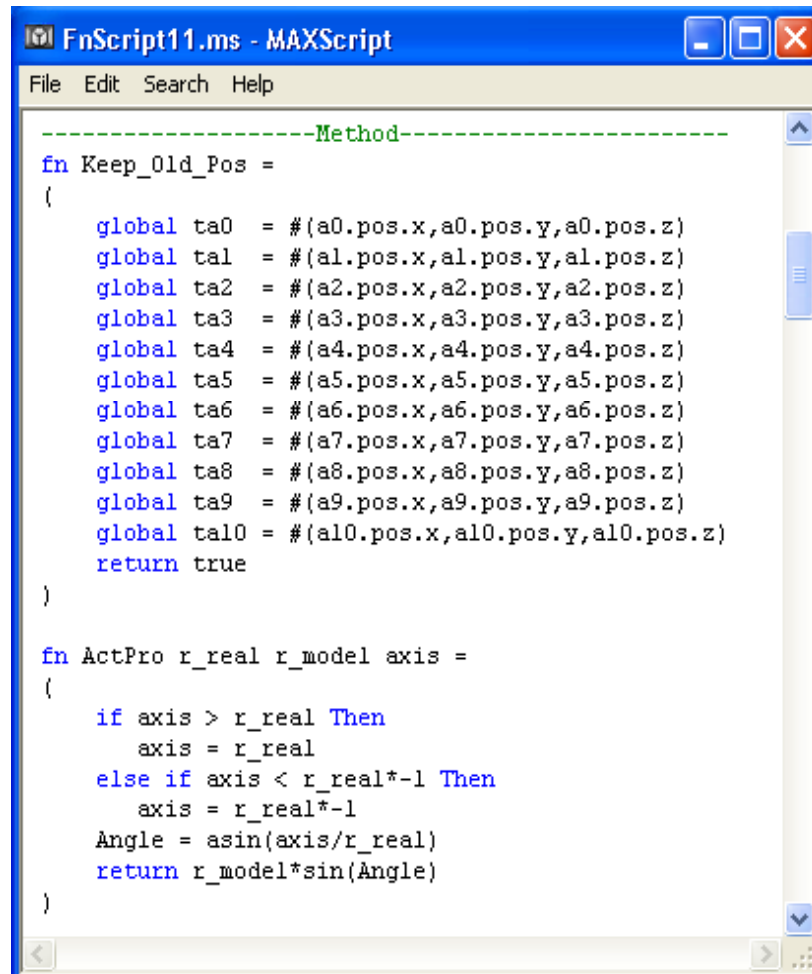
Ba0 = #(0.000000,0.000000,0.000000)
Ba1 = #(0.000000,0.000000,0.000000)
Ba2 = #(0.000000,0.000000,0.000000)
Ba3 = #(0.000000,0.000000,0.000000)
Ba4 = #(0.000000,0.000000,0.000000)
Ba5 = #(0.000000,0.000000,0.000000)
Ba6 = #(0.000000,0.000000,0.000000)
Ba7 = #(0.000000,0.000000,0.000000)
Ba8 = #(0.000000,0.000000,0.000000)
Ba9 = #(0.000000,0.000000,0.000000)
Ba10 = #(0.000000,0.000000,0.000000)

```

รูปที่ 7.7 ไฟล์สคริปต์ส่วนที่ 1

จากรูปที่ 7.7 แสดงถึงการประกาศตัวแปรเป็นแบบโกลบอล (Global) โดย a0 – a10 เป็นการประกาศอ็อบเจ็กต์ ของส่วนต่างๆของร่างกายตัวการ์ตูน 3 มิติ และ Ba0 – Ba10 เป็นการประกาศตัวแปรแบบอะเรย์ (array) เพื่อเก็บระยะทางการเคลื่อนที่ในแนวแกน X , Y และ Z





```

-----Method-----
fn Keep_Old_Pos =
(
    global ta0 = #(a0.pos.x,a0.pos.y,a0.pos.z)
    global ta1 = #(a1.pos.x,a1.pos.y,a1.pos.z)
    global ta2 = #(a2.pos.x,a2.pos.y,a2.pos.z)
    global ta3 = #(a3.pos.x,a3.pos.y,a3.pos.z)
    global ta4 = #(a4.pos.x,a4.pos.y,a4.pos.z)
    global ta5 = #(a5.pos.x,a5.pos.y,a5.pos.z)
    global ta6 = #(a6.pos.x,a6.pos.y,a6.pos.z)
    global ta7 = #(a7.pos.x,a7.pos.y,a7.pos.z)
    global ta8 = #(a8.pos.x,a8.pos.y,a8.pos.z)
    global ta9 = #(a9.pos.x,a9.pos.y,a9.pos.z)
    global ta10 = #(a10.pos.x,a10.pos.y,a10.pos.z)
    return true
)

fn ActPro r_real r_model axis =
(
    if axis > r_real Then
        axis = r_real
    else if axis < r_real*-1 Then
        axis = r_real*-1
    Angle = asin(axis/r_real)
    return r_model*sin(Angle)
)

```

รูปที่ 7.8 ไฟล์สคริปต์ส่วนที่ 2

จากรูปที่ 7.8 นี้คือการประกาศเมทอด(Method) จากรูปมีอยู่สองฟังก์ชันคือ Keep\_Old\_Pos() เป็นฟังก์ชันที่จะถูกเรียกเป็นอันดับแรก ฟังก์ชันนี้จะทำการเก็บตำแหน่งปัจจุบันของข้อต่อทุกจุด ก่อนที่จะทำการเคลื่อนไหวส่วนต่างๆของร่างกายของตัวการ์ตูนสามมิติไว้ในตัวแปรที่เป็นอะเรย์ส่วนฟังก์ชันต่อมาคือ ActPro() โดยจะมีการรับพารามิเตอร์อยู่ 3 ตัว ฟังก์ชันนี้จะทำหน้าที่ในการปรับระยะการเคลื่อนที่ให้เหมาะสมกับตัวการ์ตูนเมื่อคนที่เราทำการบันทึกการเคลื่อนไหวมา มีขนาดไม่เท่ากับตัวละคร

```

fn Root_Move =
(
    move a0[Ba0[1],Ba0[2],Ba0[3]]
)
fn Back_Bone3_Move =
(
    rr = 100.0
    rm = 30.0
    dz = tal[3] - a1.pos.z
    dy = tal[2] - a1.pos.y
    dx = tal[1] - a1.pos.x

    dx += Bal[1]
    dy += Bal[2]
    dz += Bal[3]
    dx = ActPro 100.0 60.0 dx
    dy = (rm*dy)/rr
    dz = ActPro rr rm dz
    move a1[dx,dy,dz]
)
fn Arm_Down_move =
(
    dz = ta2[3] - a2.pos.z
    dy = ta2[2] - a2.pos.y
    dx = ta2[1] - a2.pos.x
    dx += Ba2[1]
    dy += Ba2[2]
    dz += Ba2[3]
    move a2[dx,dy,dz]

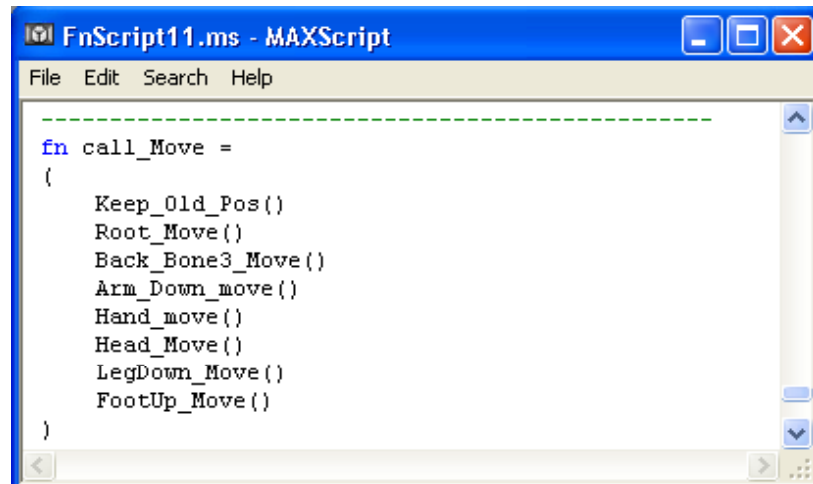
    dz = ta3[3] - a3.pos.z
    dy = ta3[2] - a3.pos.y
    dx = ta3[1] - a3.pos.x
    dx += Ba3[1]
    dy += Ba3[2]
    dz += Ba3[3]
    move a3[dx,dy,dz]
)

```

รูปที่ 7.9 ไฟล์สคริปต์ส่วนที่ 3

จากรูปที่ 7.9 แสดงฟังก์ชันที่ใช้ในการขยับส่วนต่างๆของร่างกาย จากรูป ฟังก์ชัน Root\_Move() จะทำการขยับส่วนที่เป็นรากซึ่งเป็นส่วนที่สำคัญที่สุด ต่อมาจะมาทำการขยับส่วนของกระดูกส่วนหลังคือฟังก์ชัน Back\_Bone3\_Move() ซึ่งจะมีฟังก์ชันที่ใช้ในการขยับส่วนต่างๆของร่างกาย

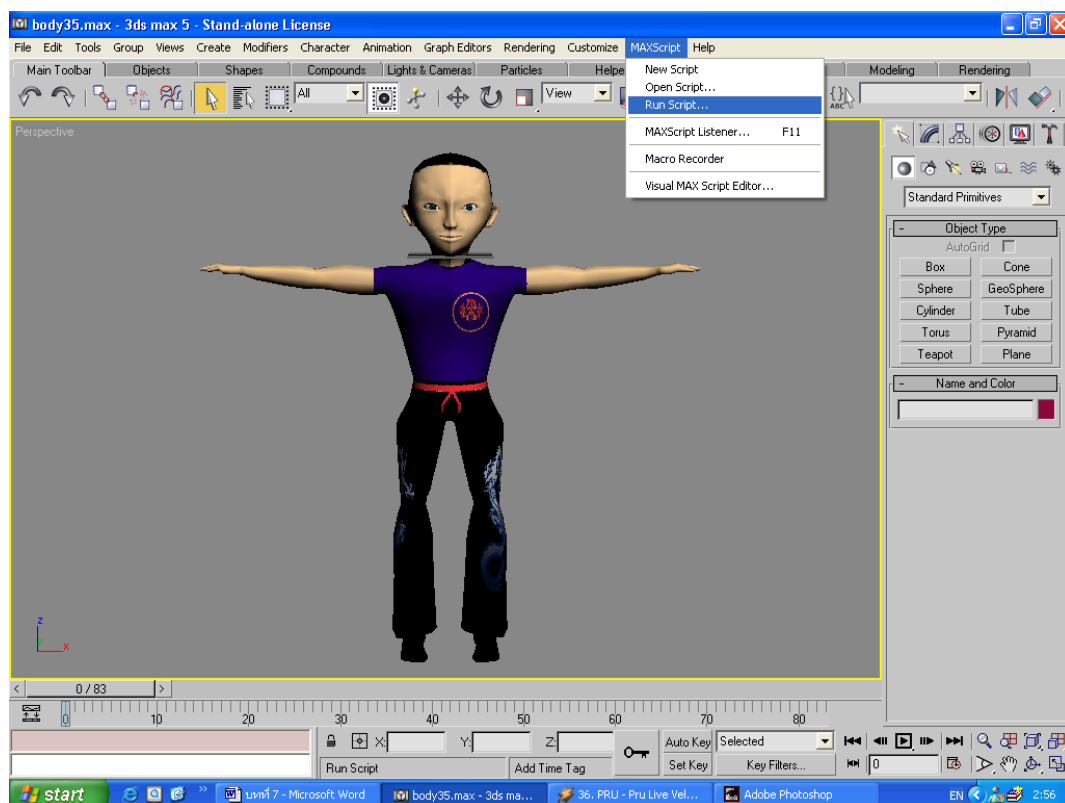
ของตัวการ์ตูน 3 มิติครบตามทุกส่วนที่เราได้ทำการติตมาครเกอร์ไว้ ซึ่งจะได้เอามาแสดงไว้ทั้งหมด ท่านสามารถ ดูโค้ดทั้งหมดของไฟล์ตัวนี้ได้ทั้งหมดที่ภาคผนวก ก และฟังก์ชันสุดท้ายที่จะขอกำลัง คล้ายๆกับเป็นเมนฟังก์ชันเพราะเป็นฟังก์ชันที่จะทำการเรียกฟังก์ชันที่กล่าวถึงมาแล้วให้ทำงานดังแสดง ไว้ดังรูปที่ 7.10



รูปที่ 7.10 ไฟล์สคริปต์ส่วนที่ 4

จากรูปที่ 7.10 จะมีฟังก์ชัน `call_Move()` โดยจะถูกเรียกใช้โดยโค้ดแมกซ์สคริปต์ที่โปรแกรมได้สร้างไว้ ที่พูดไว้แล้วในหัวข้อที่ 7.2.5 การสร้างไฟล์แมกซ์สคริปต์ โดยในฟังก์ชันนี้จะทำการเรียกฟังก์ชันอื่นๆให้ทำงานโดยลำดับการทำงานไม่สามารถสลับได้ ฟังก์ชันนี้จะถูกเรียกใช้ทุกๆครั้งที่ทำการเซ็คติย์เฟรม

ส่วนวิธีที่จะทำการรันไฟล์สคริปต์ที่โปรแกรมได้สร้างขึ้นมานั้นสามารถทำได้โดยเราต้องทำการเปิดโปรแกรมสามดี สตูดิโอ แมกซ์ แล้วทำการเปิดตัวการ์ตูน 3 มิติ ที่เราได้สร้างขึ้นมา หลังจากนั้นให้เราไปที่คลิกที่เมนูแม็กซ์สคริปต์ แล้วทำการเลือกคำสั่ง รันสคริปต์ (Run Script) ดังแสดงไว้ดังรูปที่ 7.11 แล้วเราก็ทำการเลือกไฟล์สคริปต์ที่จะนำมาทำการรันเพื่อทำเป็นภาพแอนิเมชันสามมิติ



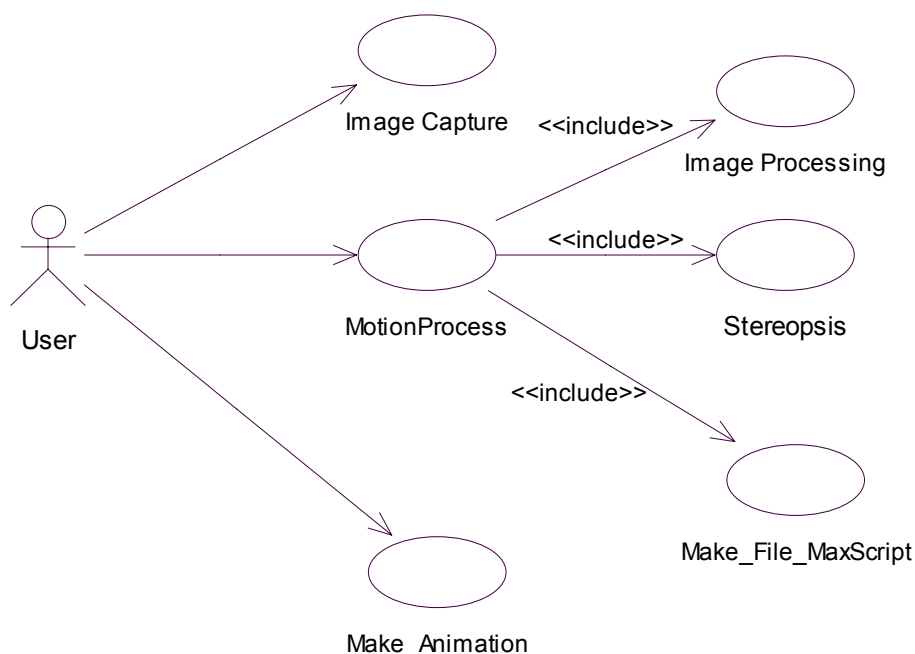
รูปที่ 7.11 แสดงวิธีการรันไฟล์แมกซ์สคริปต์

### 7.3 แผนภาพต่างๆ ของการออกแบบระบบ (Diagram)

ในที่นี้เราจะอธิบายระบบโดยใช้ไคอะแกรม ของ ยูเอ็มแอล (UML) 3 ชนิดคือ

- ยูสเคสไคอะแกรม ซึ่งใช้เพื่ออธิบายถึงผู้ใช้สามารถใช้งานใดกับระบบได้บ้าง
- ซีควเอนไคอะแกรมถูกใช้อธิบายลำดับของการติดต่อกันของอ็อบเจ็กต์
- คลาสไคอะแกรมซึ่งจะอธิบายถึงความสัมพันธ์ระหว่างคลาสที่ถูกออกแบบขึ้นมาเพื่อให้บรรลุถึงวัตถุประสงค์ของระบบ

#### 7.3.1 ยูสเคสไคอะแกรม (Use Case Diagram)



รูปที่ 7.12 แสดงยูสเคสไคอะแกรมของระบบ

จากรูปที่ 7.12 แสดงยูสเคสไคอะแกรมของระบบโดยผู้ใช้งานสามารถใช้งานระบบได้ดังนี้

- Image Capture

เป็นยูสเคสที่ติดต่อโดยผู้ใช้งาน ใช้สำหรับบันทึกการเคลื่อนไหวของสิ่งมีชีวิตที่เราสนใจ

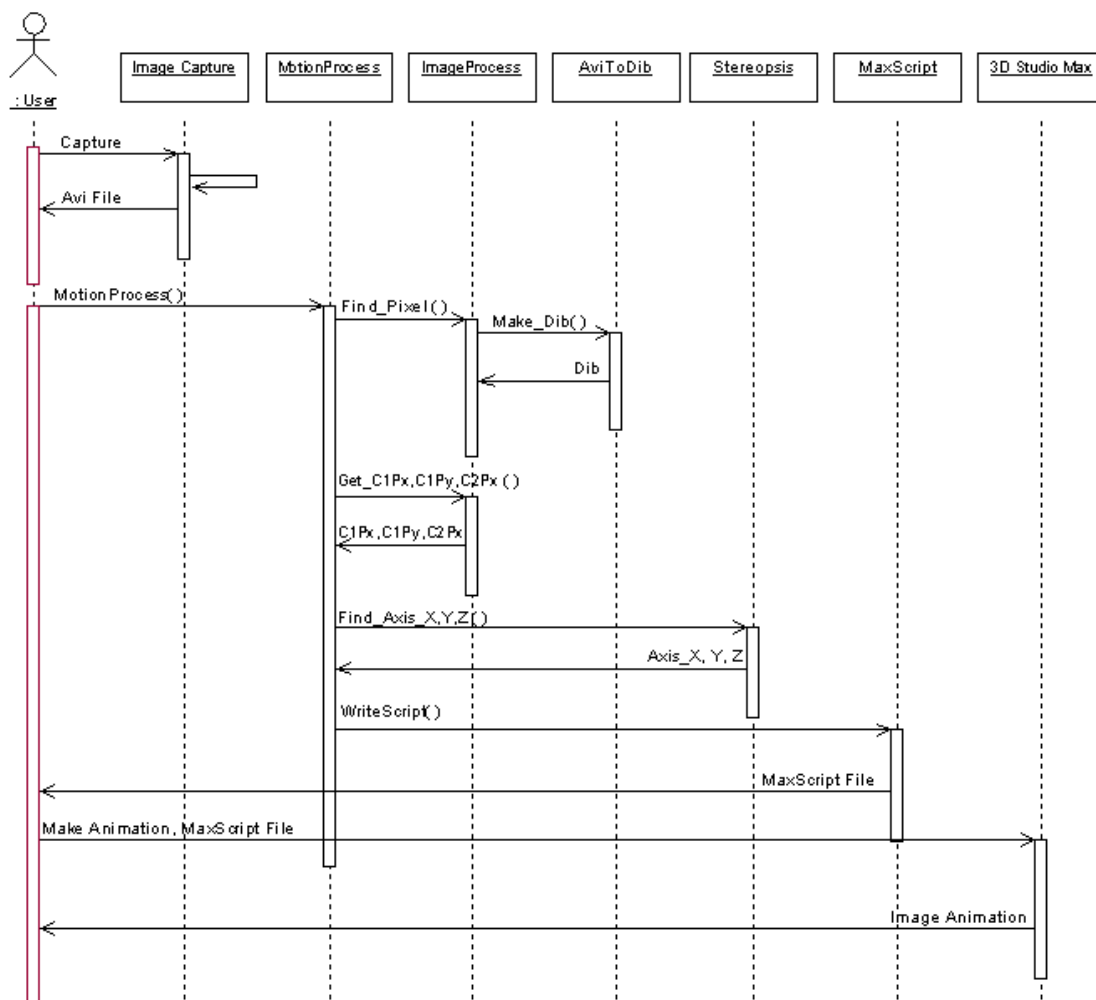
- MotionProcess

เป็นยูสเคสที่จะทำการวิเคราะห์เพื่อหาตำแหน่งในโลกจริงแล้วแปลงไปสู่โลก 3 มิติ

- Make\_Animation

เป็นยูสเคสที่ผู้ใช้งานใช้ในการสร้างภาพแอนิเมชัน ที่มีการเคลื่อนไหวได้เหมือนจริงกับการเคลื่อนไหวของสิ่งมีชีวิตต้นแบบที่เราทำการบันทึกภาพเคลื่อนไหวนั้นมา

### 7.3.2 ซีควเอนโคอะแกรม (Sequence Diagram)



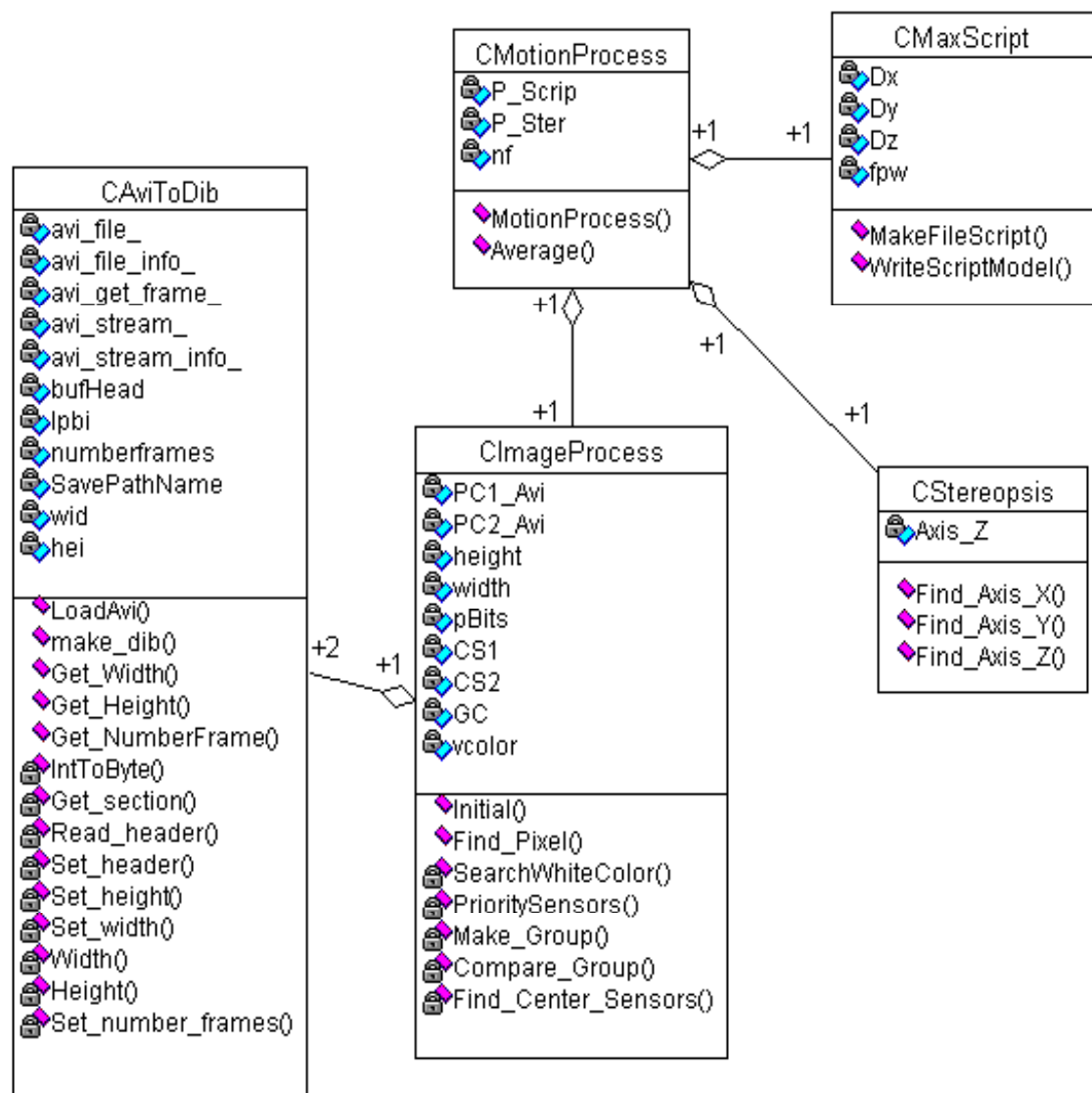
รูปที่ 7.13 แสดงซีควเอนโคอะแกรมของระบบ

จากรูปที่ 7.13 สามารถอธิบายลำดับการใช้งานของวัตถุเป็นขั้นๆ ได้ดังนี้

1. ผู้ใช้ต้องการบันทึกภาพเคลื่อนไหวของวัตถุที่สนใจจึงส่งข้อความ `Capture()` ไปยังวัตถุ `Image Capture` เพื่อทำการบันทึกภาพวัตถุที่กำลังเคลื่อนไหวแล้วส่งค่าคืนกลับมาเป็นไฟล์วิดีโอ
2. ผู้ใช้ต้องการทำ `Motion Capture` จึงส่งข้อความ `MotionProcess()` เพื่อแจ้งให้วัตถุ `MotionProcess` ทำงาน วัตถุนี้จึงส่งข้อความ `Find_Pixel` ไปยังวัตถุ `ImageProcess` เพื่อทำการหาตำแหน่งพิกเซลที่สนใจ

3. วัตถุ ImageProcess จะทำการประมวลผลต้องได้ข้อมูลของเฟรมที่ได้จากกล้องทั้งสองตัว ก่อนจึงส่งข้อความ Make\_Dib() ไปยังวัตถุ CaviToDib ให้ทำการส่งข้อมูลของ แต่ละเฟรมกลับมายังวัตถุ ImageProcess เพื่อทำการหาตำแหน่งพิกเซลต่อไป
4. วัตถุ MotionProcess ส่งข้อความ Get\_C1Px(),Get\_C1Py(),Get\_C2Px() ไปยังวัตถุ ImageProcess เพื่อให้ส่งค่าตำแหน่งพิกเซลที่คำนวณได้กลับมา
5. วัตถุ MotionProcess ส่งข้อความ Get\_Axis\_X(),Get\_Axis\_Y(),Get\_Axis\_Z() ไปยังวัตถุ Stereopsis เพื่อทำการคำนวณหาตำแหน่งที่แท้จริงของวัตถุที่เราบันทึกมาแล้วทำการแปลง ไปสู่ตำแหน่งของโลก 3 มิติ แล้วมันจะทำการส่งตำแหน่งในโลก 3 มิติที่หาได้กลับคืนมาให้
6. วัตถุ MotionProcess เมื่อได้รับข้อมูลที่ส่งกลับมาให้แล้วก็จะส่งข้อความ WriteScript() ไปให้วัตถุ แม็กส์สคลิป เพื่อทำการเขียนไฟล์แม็กส์สคลิป
7. User ได้รับไฟล์แม็กส์สคลิปแล้วก็จะทำการสร้างภาพแอนิเมชันในโลก 3 มิติโดยการส่งข้อความพร้อมกับไฟล์แม็กส์สคลิปไปยังวัตถุ 3D Studio Max เพื่อทำการสร้างภาพแอนิเมชัน ที่เหมือนกับการเคลื่อนที่ของวัตถุต้นแบบ เมื่อทำเสร็จ User ก็จะได้ภาพแอนิเมชันที่ตัวเอง ต้องการ

### 7.3.3 คลาสไดอะแกรม(Class Diagram)



รูปที่ 7.14 แสดงคลาสไดอะแกรมของโปรแกรมจับรูปแบบการเคลื่อนไหว (Motion Capture)

เราสามารถอธิบายแต่ละคลาสภายในไดอะแกรมได้ดังนี้

- คลาส CMotionProcess

มีหน้าที่ควบคุมการทำงานของคลาสต่างๆ โดยมี

1. MotionProcess() เป็นฟังก์ชันหลักในการทำงาน
2. Average() เป็นฟังก์ชันที่ใช้ในการคำนวณหาค่าผิดพลาด



#### - คลาส CImageProcess

หน้าที่ในการคำนวณหาตำแหน่งพิกเซลที่เราสนใจ โดยมีเมทอดดังนี้

1. Initial() มีหน้าที่ในการส่งชื่อไฟล์วิดีโอ ที่ได้จากการจับภาพจากกล้องทั้งสองตัว ไปให้กับคลาส CaviToDib
2. Find\_Pixel() ทำหน้าที่ดึงเอาข้อมูลของเฟรมที่ได้จากคลาส CaviToDib และทำการเรียกใช้ ฟังก์ชัน SearchWhiteColor()
3. SearchWhiteColor() ทำหน้าที่ในการหาตำแหน่งพิกเซลที่เป็นสีขาว ซึ่งเป็นสีที่เราสนใจ
4. PrioritySensors() ทำหน้าที่ในการจัดลำดับความสำคัญของมาร์เกอร์
5. Make\_Group() ทำหน้าที่ในการแบ่งกลุ่มพิกเซลที่เป็นจุดมาร์เกอร์เดียวกัน
6. Compare\_Group() ทำหน้าที่ในการเปรียบเทียบว่ามาร์เกอร์ที่หาได้นั้นเป็นมาร์เกอร์ตรงจุดใดของร่างกาย
7. Fine\_Center\_Sensors() ทำหน้าที่ในการคำนวณหาจุดศูนย์กลางของมาร์เกอร์ที่เราตรวจเช็คมาได้

#### - คลาส CaviToDib

ทำหน้าที่ในการอ่านไฟล์วิดีโอและทำการดึงเอาข้อมูลของแต่ละเฟรมออกมาโดยเฟรมพวกนี้จะถูกเก็บอยู่ในรูปของฟอร์แมตดีไอบี

1. LoadAvi() ทำหน้าที่โหลดไฟล์วิดีโอ
2. Make\_dib() ทำหน้าที่ดึงข้อมูลจากเฟรม และส่งค่ากลับไปให้ยังคลาส CImageProcess()
3. Get\_Width() ทำหน้าที่ดึงเอาค่าความกว้างของภาพ
4. Get\_Height() ทำหน้าที่ดึงเอาค่าความสูงของภาพ
5. Get\_NumberFrame() ทำหน้าที่ดึงเอาค่าจำนวนเฟรมทั้งหมดของไฟล์วิดีโอ จาก Header ของไฟล์

#### - คลาส Cstereopsis

ทำหน้าที่คำนวณหาค่าคาทิสเียนคอดิเนต

1. Find\_Axis\_X() คำนวณหาตำแหน่งแกน 'X' ของวัตถุในโลกจริง
2. Find\_Axis\_Y() คำนวณหาตำแหน่งแกน 'Y' ของวัตถุในโลกจริง
3. Find\_Axis\_Z() คำนวณหาตำแหน่งแกน 'Z' ของวัตถุในโลกจริง

#### - คลาส CmaxScript

1. MakeFileScript() ทำหน้าที่สร้างไฟล์ MaxScript (\*.ms)
2. WriteScriptModel() ทำหน้าที่เขียนไฟล์ แม็กส์สคริป